

Lecture 2: Algorithmic Thinking, Recursion, and Bubble Sort

Harsh Parikh

hp537@yale.edu

BIS 527: Introduction to Health Data Science

Yale School of Public Health

Lecturer: Harsh Parikh • Fall 2026

TL;DR

Putting records in order is among the most common tasks in computing, and the cost of performing it is worth knowing exactly. Ordering ten thousand records is a task no single step performs, so it must be broken down. This chapter develops that idea through one procedure: bubble sort, written recursively. It then asks the three questions this course asks of every procedure. Does it halt? Is it correct? What does it cost? The answers give the measure argument for halting, weak and strong induction for correctness, and a recurrence for cost.

AI use declaration. We used Claude Opus 5 to research as well as polish the writing in the notes.

1 Motivation

A clinic holds the records of n patients seen last month, each with a height recorded in centimetres, and the task is to arrange those records in order from shortest to tallest.

Estimating the empirical distribution function of the cohort is one task that requires exactly this. That function is $\hat{F}(x) := \frac{1}{n} \sum_{i=0}^{n-1} \mathbf{1}\{A[i] \leq x\}$, where $\mathbf{1}\{\cdot\}$ takes the value one when the statement inside it holds and zero otherwise, so $\hat{F}(x)$ is the proportion of the cohort no taller than x . It is a step function that rises by $1/n$ at each recorded height, and a list of the heights in order gives it outright, because the k -th smallest height is the location of the k -th step. The ordered list is not a means to the answer here. It is the answer.

A person can put ten records in order by hand and cannot do it for ten thousand, so we write a procedure instead. Writing one is easy, but knowing that what we wrote is right is not, and neither is knowing what it will cost when the cohort grows.

2 Setup and Notation

Prerequisites. This chapter assumes summation notation and proof by induction, states both forms of induction it uses in Section 5, and assumes no prior knowledge of any sorting procedure.

Notation. Let A be a list holding n records, indexed from zero as $A[0], \dots, A[n-1]$, and write $A[i:j]$ for the slice $A[i], \dots, A[j-1]$, which is empty when $i \geq j$. Indices start at zero and slices exclude their right end, as in Python, so the last pair of neighbours in a list of n records is $A[n-2]$ and $A[n-1]$. Cormen et al. (2022) index from one instead, and write $A[1..n]$ for a whole array and $A[p..r]$ for a subarray, so a reader moving between that book and this chapter should shift every index by one. The records are drawn from a *totally ordered set*, by which we mean that for any two records x and y exactly one of $x < y$, $x = y$, $x > y$ holds and the relation is transitive. Height recorded in whole centimetres is such a set.

Definition 1 (The sorting problem). **Input:** a list A of n records drawn from a totally ordered set. **Output:** a *permutation*, that is a reordering, of those same n records, arranged so that $A[0] \leq A[1] \leq \dots \leq A[n-1]$.

The word permutation carries a condition that is easy to overlook, namely that the records which come out must be the records that went in. A procedure returning $\langle 1, 2, 2 \rangle$ when given $\langle 3, 1, 2 \rangle$ has produced an ordered list that is not a permutation of its input, and it has not sorted anything.

Cost. We count *comparisons*, one for each test of a record against another, rather than seconds, because seconds depend on the machine and the language rather than on the method. We count *space* as the memory used beyond the list itself.

The three questions. Of every procedure in this course we ask, in this order, whether it halts, whether it is correct, and what it costs in time and in space. The order matters, because a procedure that does not halt has no output to be correct about.

3 Algorithmic Thinking, and Recursion

In this section, we turn the task of ordering a cohort into a procedure that calls itself.

Decomposition. An *algorithm* is a finite sequence of unambiguous steps that turns an input into an output, and algorithmic thinking is the practice of producing such a sequence for a stated task. When no single step performs the task, the task is broken into subtasks, each subtask is solved, and the solutions are combined. Subtasks come in two kinds, which differ in whether the subtask

resembles the original task, and the decomposition below is of the second kind, producing smaller instances of the task we started with.

One sweep. Stand the patients of Section 1 in a row, to be put in height order, and allow only one operation: compare two neighbours, and exchange them if the left one is taller. Sweep once from left to right. The tallest person takes part in a comparison at every position from their starting position to the end and is exchanged rightward at each, so they finish at the right-hand end. What that sweep leaves behind is the point of the construction. The last position holds the largest record and never needs to move again, so what remains is a list of $n - 1$ records to be put in order, which is the original problem on a smaller input and can be handed back to the same procedure.

Definition 2 (Recursive procedure). A procedure is *recursive* if it solves a problem by reducing it to smaller instances of the same problem and calling itself on them. A *base case* is an input the procedure answers outright, without calling itself, and a *reduction step* turns any other input into smaller ones and calls the procedure on them.

Listing 1: Bubble sort, written recursively.

```
1 def sort(A, n):                                # sort A[0:n] in place
2     if n <= 1:                                  # base case: nothing to do
3         return
4     moved = bubble_up(A, 0, n)                  # one sweep
5     if not moved:                                # already in order
6         return
7     sort(A, n - 1)                              # the same problem, one smaller
8
9 def bubble_up(A, j, n):                         # True if it exchanged anything
10     if j >= n - 1:                              # base case: no pair left
11         return False
12     swapped = False
13     if A[j] > A[j + 1]:
14         A[j], A[j + 1] = A[j + 1], A[j]
15         swapped = True
16     rest = bubble_up(A, j + 1, n)
17     return swapped or rest
```

Both procedures in Listing 1 have one base case and one reduction step, and writing them in this form is not a matter of taste, because it makes the proof of correctness an induction and the cost a recurrence.

4 Halting

We now ask the first of the three questions: does the procedure stop? A recursive procedure is defined in terms of itself, so nothing in the definition guarantees that a chain of calls ever ends, and three separate failures are possible.

Three failures. The procedure `def f(k): return f(k-1)` answers no input outright, so every call makes another call and a base case is necessary. The procedure `def g(k): if k == 0: return 0; else: return g(k)` has a base case at $k = 0$, but on $k = 3$ every call in the chain is again `g(3)`, so the base case exists while nothing moves the argument toward it, and the argument must therefore change at every call. The procedure `def h(k): if k == 0: return 0; else: return h(k-2)` has a base case and changes its argument, yet on $k = 5$ the arguments run $5, 3, 1, -1, -3, \dots$ and never equal 0, because the change must reach the base case rather than pass it. All three requirements are met at once by insisting on a quantity that cannot decrease forever.

Definition 3 (Measure). A *measure* for a recursive procedure is a quantity computed from its arguments that is a non-negative integer at every call, and that strictly decreases at every recursive call the procedure makes.

A strictly decreasing sequence of non-negative integers is finite, so a procedure with a measure makes finitely many nested calls and a chain of calls can end only at a base case. Each condition rules out one of the failures above, since the argument of `g` does not decrease and the argument of `h` decreases without being bounded below.

Theorem 1 (Halting). *For every list A and every $n \geq 0$, the call `sort(A, n)` halts.*

Proof. For `bubble_up` take $d := n - j$, the number of cells left in the slice, which is an integer, is non-negative whenever the procedure is called with $j \leq n$, and is replaced by $d - 1$ at the single recursive call, since that call replaces j by $j + 1$. By Definition 3 it is a measure, so `bubble_up` halts.

For `sort` take the argument n itself, which is a non-negative integer that the single recursive call `sort(A, n-1)` decreases by exactly one, so it cannot pass the base case at $n \leq 1$. The call to `bubble_up` halts by the previous paragraph, and therefore `sort` halts. $\square$

This is the property the course calls *completeness*: the procedure halts, and the records it returns are the records it was given. The second half of that claim is proved in Section 5.

5 Correctness

We now show that the procedure returns the required output on every input, and not merely on the inputs we happen to try. A test shows that a procedure works on the inputs tested and says nothing

about the inputs not tested, of which there are infinitely many, whereas a procedure defined in terms of itself can be proved in terms of itself.

Certificates. Call a proof that the procedure returns the required output on every input of size k a *certificate* for size k . The reduction step turns an input of size n into smaller inputs, so if the work it performs is enough to turn certificates for those smaller inputs into a certificate for size n , and if we hold a certificate for the base case, then we hold one for every size. That argument is induction, and it comes in two forms.

Definition 4 (Weak and strong induction). Let $P(n)$ be a claim about each non-negative integer n . *Weak induction*: if $P(0)$ holds, and $P(n - 1)$ implies $P(n)$ for every $n \geq 1$, then $P(n)$ holds for every n . *Strong induction*: if $P(0)$ holds, and the assumption that $P(k)$ holds for every $k < n$ implies $P(n)$, then $P(n)$ holds for every n .

Matching the shape of the procedure. The two forms differ in which certificates the hypothesis grants, since weak induction grants the certificate for the immediate predecessor $n - 1$ and strong induction grants those for every $k < n$ at once. The hypothesis must cover exactly the calls the procedure makes, and $\text{sort}(A, n)$ calls $\text{sort}(A, n-1)$ and nothing else.

Lemma 2 (What one sweep achieves). Let $0 \leq j \leq n - 1 < |A|$ and let V be the values held in $A[j:n]$ on entry. After $\text{bubble_up}(A, j, n)$ returns, $A[j:n]$ holds V , and $A[n - 1] = \max V$.

In words, one sweep rearranges the slice without creating or losing a value, and leaves the largest value of the slice in the last cell.

Proof. Induction on $d = n - j$.

Base case ($d = 1$). Here $j = n - 1$, the procedure returns at once, and $A[n - 1]$ is the maximum of the one-element slice.

Inductive hypothesis. Assume the claim for the call $\text{bubble_up}(A, j+1, n)$, whose slice has $d - 1$ cells.

Inductive step ($d \geq 2$). The procedure compares $A[j]$ with $A[j + 1]$ and exchanges them if they are out of order, so afterwards $A[j + 1]$ holds the larger of the two while the pair holds the same two values. It then calls $\text{bubble_up}(A, j+1, n)$, which by the hypothesis leaves $A[n - 1]$ holding the maximum of $A[j+1:n]$ and permutes that slice. That maximum is the larger of the original $A[j]$ and $A[j + 1]$, or a value in the original $A[j+2:n]$, whichever is greatest, and in either case it is $\max V$. Every write performed is one half of an exchange, so no value is created and none is lost. □

Theorem 3 (Correctness). *For every list A and every $n \geq 0$, after $\text{sort}(A, n)$ returns, the slice $A[0:n]$ holds the values it held on entry, counted with multiplicity, and $A[0] \leq A[1] \leq \dots \leq A[n-1]$.*

Proof. Weak induction on n .

Base case ($n \leq 1$). A list of at most one record is in non-decreasing order, and the procedure returns without writing to it.

Inductive hypothesis. Assume the claim for $\text{sort}(A, n-1)$.

Inductive step ($n \geq 2$). The call $\text{bubble_up}(A, 0, n)$ permutes $A[0:n]$ and leaves $\max A[0:n]$ in $A[n-1]$, by Lemma 2. If it reports no exchange then $A[j] \leq A[j+1]$ held at every position, so the list was already in order and returning is correct. Otherwise the procedure calls $\text{sort}(A, n-1)$, which by the hypothesis permutes $A[0:n-1]$ and leaves it in non-decreasing order without writing to $A[n-1]$. Every value in $A[0:n-1]$ is at most $A[n-1]$, so the whole slice is in non-decreasing order, and a composition of permutations is a permutation. $\square$

The theorem gives both halves of Definition 1: the records that come out are the records that went in, and they come out in non-decreasing order.

Remark 1 (Where the strong form is required). Consider the variant that sweeps twice before recursing, left to right placing $\max A[0:n]$ at position $n-1$, then right to left placing $\min A[0:n-1]$ at position 0, and then calls itself on the remaining slice $A[1:n-1]$ of $n-2$ records. Halting is unchanged in form, with base cases at $n=0$ and $n=1$ to catch both parities, but correctness is not, because the inductive step needs the certificate for $n-2$, which weak induction does not grant and strong induction does. Any procedure whose calls are on sizes other than $n-1$ requires the strong form for the same reason.

6 Cost

It remains to count what the procedure costs. Let $T(n)$ be the number of comparisons $\text{sort}(A, n)$ makes in the worst case. One sweep over n records compares $n-1$ neighbouring pairs and the remaining work is a sweep over $n-1$ records, so

$$T(n) = T(n-1) + (n-1) \quad (n \geq 2), \quad T(1) = 0,$$

and unrolling the recurrence gives

$$T(n) = (n-1) + (n-2) + \dots + 1 = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}.$$

The worst case therefore costs exactly $n(n-1)/2$ comparisons. The best case is far cheaper. If

the list arrives already in order the first sweep exchanges nothing, `bubble_up` reports `False`, and `sort` returns after exactly $n - 1$ comparisons.

Space is the second currency, and the standard term for spending almost none of it is due to [Cormen et al. \(2022\)](#): a sorting procedure *sorts in place* if only a constant number of records are ever stored outside the list. Each exchange rewrites two cells and needs no other storage, so the sweep itself holds a constant amount beyond the list. The calls, however, are nested $n - 1$ deep, so at the deepest point $n - 1$ suspended calls are held at once and the recursive form does not sort in place. Rewriting both procedures as loops removes that cost and leaves the comparison count unchanged.

7 Worked Example

Take the recorded heights of five patients, in centimetres: $A = \langle 171, 158, 180, 152, 163 \rangle$. Each row below is the list after one sweep.

sweep	list after the sweep	comparisons	exchanged
1	$\langle 158, 171, 152, 163, 180 \rangle$	4	yes
2	$\langle 158, 152, 163, 171, 180 \rangle$	3	yes
3	$\langle 152, 158, 163, 171, 180 \rangle$	2	yes
4	$\langle 152, 158, 163, 171, 180 \rangle$	1	no

The total is $4 + 3 + 2 + 1 = 10$ comparisons, which is $n(n - 1)/2$ at $n = 5$, and running the procedure on all 120 orderings of these five values gives no ordering costing more than 10, so this input attains the worst case. After sweep 1 the largest value, 180, is in its final position and after sweep 2 the next largest is in place, and the fourth sweep exchanges nothing, so `sort` returns without a fifth. The fourth row is the output, holding the five records the procedure was given, arranged from shortest to tallest.

8 Beyond the Lecture

The proofs here are inductions because the procedures are recursive. Written as loops, the same procedures call instead for *loop invariants*, a claim true before the loop starts, preserved by each iteration, and strong enough at exit to give the result, whose three parts [Cormen et al. \(2022\)](#) name initialization, maintenance and termination.

Exact counts become impractical for longer procedures, and the standard remedy, which the lecture did not cover, is to report only the rate at which a count grows with n . For functions f and g from the positive integers to the non-negative reals, write $f(n) \in \Omega(g(n))$ when there exist positive constants c and n_0 with $0 \leq c g(n) \leq f(n)$ for all $n \geq n_0$, and $f(n) \in \Theta(g(n))$ when there exist positive constants c_1, c_2 and n_0 with $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$. A

function in $\Theta(g(n))$ is held between two constant multiples of g once n is large enough, so Θ records a rate of growth and discards the constants that depend on the machine, and [Cormen et al. \(2022\)](#) write $f(n) = \Theta(g(n))$ where we write $\in$. In this notation the worst case of bubble sort is $n(n-1)/2 \in \Theta(n^2)$, the best case is $\Theta(n)$, and the memory held by suspended calls is $\Theta(n)$. The decomposition used here removes one record per call and pays $\Theta(n^2)$ for it. A decomposition that splits the list into two halves, orders each, and combines two ordered halves in one pass satisfies $T(n) = 2T(n/2) + \Theta(n)$, which is $\Theta(n \log_2 n)$, a growth rate that [Cormen et al. \(2022\)](#) write as $\Theta(n \lg n)$, using $\lg$ throughout for the base-two logarithm. That pattern of dividing a problem, conquering the parts by recursion, and combining their solutions is the *divide-and-conquer* paradigm. A *comparison sort* gains information about the records only by comparing pairs of them, and every comparison sort requires $\Omega(n \log_2 n)$ comparisons in the worst case ([Cormen et al., 2022](#)), so merge sort is asymptotically optimal among them and bubble sort is not. The name “bubble sort” and the question of whether it should be taught at all have their own small literature ([Astrachan, 2003](#)).

9 Summary

The decomposition used here produces a smaller instance of the task itself, since one sweep puts the largest record in its final position and leaves $n-1$ records to be put in order, and a procedure built that way needs both a base case and a reduction step.

Three questions settle the procedure. It halts, because n is a non-negative integer that strictly decreases by one at every call, and such a quantity can neither decrease forever nor step past the base case. It is correct by an induction whose hypothesis covers the call the procedure makes, and weak induction suffices because that call is on $n-1$, while a procedure calling itself on other sizes needs the strong form. It costs exactly $n(n-1)/2$ comparisons in the worst case and $n-1$ in the best, and the recursive form holds $n-1$ suspended calls at its deepest point.

Further Reading

[Cormen et al. \(2022\)](#) give the loop-invariant treatment of insertion sort, the divide-and-conquer analysis of merge sort, the lower bound for comparison sorting, and the correctness of bubble sort as a problem. [Knuth \(1998\)](#) covers bubble sort’s exact comparison counts.

References

- Astrachan, O. (2003). Bubble sort: An archaeological algorithmic analysis. *ACM SIGCSE Bulletin*, 35(1):1–5.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4th edition.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2nd edition.